CS 320 Project Two: Summary and Reflections Report

Ilianna Chavez

Summary: Unit Testing Approach

For Project One, I worked on the contact, task, and appointment services by turning the requirements into individual JUnit tests. I found it easier to work through one rule at a time instead of just checking whether the program ran. In the contact service, I checked the ID, names, phone number, and address, including null values and entries that were too long. I also tested adding contacts, rejecting duplicate IDs, deleting contacts, and updating their information. For example, ContactTest.java lines 72–93 check invalid phone numbers, while ContactServiceTest.java lines 84–94 verify a phone update.

I followed a similar process for tasks. TaskTest.java checks the ID, name, and description limits, and TaskServiceTest.java checks adding, deleting, and updating tasks. The duplicate-ID test in TaskServiceTest.java lines 16–26 expects an IllegalArgumentException when a second task uses the same ID. For appointments, I checked that dates were not null or in the past, descriptions met the limit, and duplicate IDs were rejected. AppointmentTest.java lines 38–51 cover invalid dates, while AppointmentServiceTest.java lines 22–41 cover duplicate appointments. These tests were tied directly to the rules we were given.

My ZIP contains 47 JUnit test methods: 19 for contacts, 15 for tasks, and 13 for appointments. In my earlier journal, I reported that all 15 task tests passed with zero failures or errors. That gives me confidence in those tested behaviors, but passing tests and a test count are not the same as a measured coverage percentage. I did not run a coverage tool such as JaCoCo, so I cannot claim an exact percentage. I would use one in a future project to see which lines and branches still need attention.

Summary: Writing the JUnit Tests

Writing the tests taught me that code compiling does not automatically mean it is correct. I used assertEquals to compare actual values with what I expected. For example, TaskServiceTest.java line 53 checks that an updated task name is “Study,” and line 63 checks that an updated description is “Assignment completed.” I also used assertThrows for invalid input and duplicate IDs. This made the tests more meaningful because each one had a clear expected outcome.

I tried to keep the tests focused and efficient. Most tests create their own service object and only the data needed for that case, so earlier tests do not affect later ones. I did not need a database or a full application to test these in-memory services. I also learned that a test can be improved even if it passes: for example, my delete tests use assertDoesNotThrow, which checks that deleting succeeds but does not independently confirm the item is gone. Adding a follow-up check would make those tests stronger.

Reflection: Testing Techniques

The main technique I used was automated unit testing with JUnit. I also used positive and negative test cases: positive tests checked normal inputs, and negative tests checked values the application should reject. Testing one class or service behavior at a time helped me narrow down where a problem might be. These techniques work especially well for small components with clear requirements and make regression testing easier after changes.

I did not do full integration, system, or performance testing. Integration testing would matter if these services had to communicate with a database or other application components. System testing would check whether the completed mobile application works from beginning to end. Performance testing would become important if many users or large amounts of data were

involved. None of those techniques replace unit testing; they answer different questions as a project grows.

Reflection: Mindset and Professional Responsibility

I learned to be more cautious about assuming that the happy path is enough. Contact with a valid phone number is only part of the story; I also needed to see what happened with a short phone number or letters where digits belong. Appointments made me think about dates as well. A change in validation could affect both object creation and later updates, so those related behaviors deserve separate tests. I also see room to improve by testing exact boundary values and checking whether a deleted record can still be found.

Testing my own code can create bias because I already know how I intended it to work. To challenge that, I wrote tests expecting failures, such as duplicate task IDs and appointments with past dates. I tried to use the requirements as my checklist instead of treating my assumptions as proof. In future projects, I would also ask someone else to review the tests because another person may notice cases I missed.

Being disciplined about quality means not stopping as soon as the code runs once. Small shortcuts, such as skipping invalid-input tests or leaving unclear code behind, can turn into technical debt when someone must change the software later. My plan is to keep tests small, run them after changes, name them clearly, review uncovered cases, and fix issues while they are still manageable. Before this course, I mostly thought testing happened after coding. Now I understand why it should be part of development from the beginning.

References

Chavez, I. (2026). CS 320 Project One: Contact, task, and appointment service source code and JUnit tests [Unpublished course project]. Southern New Hampshire University.